For simplicity, we assume that in the beginning, the crooked dealer is equally likely to use the fair or biased coin, an assumption that is modeled by setting $\Pr(\pi_0 \rightarrow \pi_1) = 1/2$, where π_0 is a "silent" initial state that does not emit any symbols. The probability of π is equal to the product of its transition probabilities,

$$\Pr(\pi) = \prod_{i=1}^{n} \Pr(\pi_{i-1} \rightarrow \pi_i) = \prod_{i=1}^{n} transition_{\pi_{i-1},\, \pi_i}.$$

Probability of a Hidden Path Problem: *Compute the probability of an HMM's hidden path.*

Input: A hidden path π in an HMM (Σ, *States*, *Transition*, *Emission*).

Output: The probability of this path, $\Pr(\pi)$.

CODE CHALLENGE: Solve the Probability of a Hidden Path Problem.

Given: A hidden path π followed by the states *States* and transition matrix *Transition* of an HMM (Σ, *States*, *Transition*, *Emission*).

Return: The probability of this path, $Pr(\pi)$.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/hmm/ProbabilityOfHiddenPath.txt)

Sample Input:

```
ABABBBAAAA
--------
A B
--------
        A       B
A       0.377   0.623
B       0.26    0.74
```

Sample Output:

```
0.000384928691755
```

Time Limit: 5 mins

To solve this problem please visit stepic.org/lesson/-11594/step/2

To compute $\Pr(x|\pi)$ for a general HMM, we will write $\Pr(x_i|\pi_i)$ to denote the emission probability $emission_{\pi_i}(x_i)$ that symbol x_i was emitted given that the HMM was in state π_i. As a result, for a given path π, the HMM emits a string x with probability equal to the product of emission probabilities along that path,

$$\Pr(x|\pi) = \prod_{i=1}^{n} \Pr(x_i|\pi_i) = \prod_{i=1}^{n} emission_{\pi_i}(x_i).$$

Probability of an Outcome Given a Hidden Path Problem: *Compute the probability that an HMM will emit a string given its hidden path.*

Input: A string x emitted by an HMM (Σ, *States*, *Transition*, *Emission*) and a hidden path π.

Output: The conditional probability $\Pr(x|\pi)$ that x will be emitted given that the HMM follows the hidden path π.

CODE CHALLENGE: Solve the Probability of an Outcome Given a Hidden Path Problem.

Input: A string *x,* followed by the alphabet from which *x* was constructed, followed by a hidden path *π,* followed by the states *States* and emission matrix *Emission* of an HMM (*Σ*, *States*, *Transition*, *Emission*).

Output: The conditional probability Pr(*x*|*π*) that *x* will be emitted given that the HMM follows the hidden path *π*.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/hmm/ProbabilityOfOutcomeGivenHiddenPath

Sample Input:

```
zzzyxyyzzx
--------
x y z
--------
BAAAAAAAAA
--------
A B
--------
        x       y       z
A       0.176   0.597   0.228
B       0.225   0.572   0.203
```

Sample Output:

```
3.61562811615e-06
```

Time Limit: 5 mins

To solve this problem please visit
stepic.org/lesson/-11594/step/4

Decoding Problem: *Find an optimal hidden path in an HMM given a string of its emitted symbols.*

Input: A string x emitted by an HMM (Σ, *States*, *Transition*, *Emission*).

Output: A path π that maximizes the (unconditional) probability $\Pr(x, \pi)$ that x will be emitted given that the HMM follow over all possible paths through this HMM.

We will apply a dynamic programming algorithm to solve the Decoding Problem. First, define $s_{k,i}$ as the product weight of an optimal path (i.e., a path with maximum product weight) from *source* to node (k, i). The **Viterbi algorithm** relies on the fact that the first $i-1$ edges of an optimal path from *source* to (k, i) must form an optimal path from *source* to (l, $i-1$) for some (unknown) state l. This observation yields the following recurrence:

$$\begin{aligned} s_{k,\,i} &= \max_{\text{all states } l} \left\{ s_{l,\,i-1} \cdot (\text{weight of edge between nodes}(l, i-1) \text{ and } (k, i)) \right\} \\ &= \max_{\text{all states } l} \left\{ s_{l,\,i-1} \cdot Weight_i(l, k) \right\} \\ &= \max_{\text{all states } l} \left\{ s_{l,\,i-1} \cdot transition_{\pi_{i-1},\,\pi_i} \cdot emission_{\pi_i}(x_i) \right\} \end{aligned}$$

For i = 1, every node in the leftmost column is connected to *source*, so that the above recurrence becomes

$$\begin{aligned} s_{k,\,1} &= s_{source} \cdot \ (\text{weight of edge between source and } (k, 1)) \\ &= s_{source} \cdot Weight_0(source, k) \\ &= s_{source} \cdot transition_{source,\,k} \cdot emission_k(x_1) \end{aligned}$$

In order to initialize this recurrence, we set s_{source} equal to 1. We can now compute the maximum product weight over all paths from *source* to *sink* as

$$s_{sink} = \max_{\text{all states } l} s_{l,\,n}.$$

CODE CHALLENGE: Implement the Viterbi algorithm solving the Decoding Problem.

Input: A string *x*, followed by the alphabet from which *x* was constructed, followed by the states *States*, transition matrix *Transition*, and emission matrix *Emission* of an HMM (Σ, *States*, *Transition*, *Emission*).

Output: A path that maximizes the (unconditional) probability Pr(*x*, *π*) over all possible paths *π*.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/hmm/ViterbiAlgorithm.txt)

Sample Input:

```
xyxzzxyxyy
--------
x y z
--------
A B
--------
        A       B
A       0.641   0.359
B       0.729   0.271
--------
        x       y       z
A       0.117   0.691   0.192
B       0.097   0.42    0.483
```

Sample Output:

```
AAABBAAAAA
```

Time Limit: 5 mins

To solve this problem please visit

stepic.org/lesson/-11594/step/6

Outcome Likelihood Problem: *Find the probability that an HMM emits a given string.*

Input: A string *x* emitted by an HMM (*Σ*, *States*, *Transition*, *Emission*).

Output: The probability Pr(*x*) that the HMM emits *x*.

We denote the total product weight of all paths from *source* to node (k, i) in the Viterbi graph as $forward_{k,\,i}$; note that $forward_{sink}$ is equal to Pr(*x*). To compute $forward_{k,i}$, we will divide all paths connecting *source* to (*k, i*) into $|States|$ subsets, where each subset contains those paths that pass through node (*l, i* - 1) (with product weight $forward_{l,i-1}$) before reaching (*k, i*) for some *l* between 1 and $|States|$. Therefore, $forward_{k,\,i}$ is the sum of $|States|$ terms:

$$\begin{aligned} forward_{k,\,i} &= \sum_{\text{all states } l} forward_{l,\,i-1} \cdot \text{(weight of edge connecting } (l, i-1) \text{ and } (k, i)) \\ &= \sum_{\text{all states } l} forward_{l,\,i-1} \cdot Weight_i(l, k) \end{aligned}$$

Note that the only difference between the above recurrence and the Viterbi recurrence is that the maximization in the Viterbi algorithm has changed into a summation symbol. We can now solve the Outcome Likelihood Problem by computing $forward_{sink}$, which is equal to

$$\sum_{\text{all states } k} forward_{k,\,n}.$$

CODE CHALLENGE: Solve the Outcome Likelihood Problem.

Input: A string *x*, followed by the alphabet from which *x* was constructed, followed by the states *States*, transition matrix *Transition*, and emission matrix *Emission* of an HMM (*Σ*, *States*, *Transition*, *Emission*).

Output: The probability Pr(*x*) that the HMM emits *x*.

Extra Dataset (http://bioinformaticsalgorithms.com/data/extradatasets/hmm/OutcomeLikelihood.txt)

Sample Input:

```
xzyyzzyzyy
--------
x y z
--------
A B
--------
        A       B
A       0.303   0.697
B       0.831   0.169
--------
        x       y       z
A       0.533   0.065   0.402
B       0.342   0.334   0.324
```

Sample Output:

```
1.1005510319694847e-06
```

Time Limit: 5 mins

To solve this problem please visit

stepic.org/lesson/-11594/step/8